The Evolution of Ethernet 1983-2023

"Ethernet doesn't work in theory, It only works in practice" – *Bob Metcalf*

"One concern I would have (and many others must have too) is how the plateauing of semiconductor device scaling (i.e. slowing of Moore's Law) may hinder future networking speed ups (thus the recent emergence of parallel links). It is nevertheless remarkable how far it has advanced!!" – *Robert Garner*

Ganging together parallel links may not an optimal way to use this resource. Parallel links 'between routers' benefits from 'aggregated bandwidth', which has marketing value, but it has downsides:

- Aggregating multiple links combines what should be independent failure domains and entwines them in a common one (single point of failure).
- Multiple links appears to address the link bottleneck problem but it makes them even more sensitive to congestion and retry storms, and other cascade failures as flows all self-organize into power-laws, with unexpected failures becoming inevitably more frequent.
- An alternative solution is to use the links independently for 'microservice interaction sets'. Those different 'interaction sets' are graphs which overwhelmingly communicate east-west in datacenters, rather than the north-south prevailing paradigm.
- In 'directly connected' mesh networks, 'graph bandwidth' is a better metric than 'max-flow min-cut'.
- There are other opportunities to make important upgrades by re-examining the nature of the A2A (Any to Any) communication paradigm used in both L2 and L3 protocols today. We know from advancements in Graph Theory (Hypergraphs) that we can provide much better 'confinement' guarantees.
- It makes sense to still use Ethernet frames, and much of the Ethernet philosophy, as a foundation.

Stop and Wait (SAW) Unit of Flow Control, Not Pause Frames

A design choice in Slice-Engine.pdf is what is the 'unit' of Stop And Wait (SAW) before the other side can send another unit? Choices include 1 slice, n-slices, 1 frame (64 bytes, or 8 slices of 64-bits), or 2 frames.

This question can be resolved by recognizing that the number of 'bits on the wire' is a function of the length of the wire (and in our case – with short-reach connections between cells in a rack), this is much smaller than typical cables in a legacy cabling environment. For example, 5cm, 50 cm, or 500cm might be relevant 'within a rack', and 1m to 5m between racks.

A simple calculation yields, at 10Gbit/s, 10 bits arrive each ns, and at 100 Gbit/s, 100 bits arrive each ns. The SERDES Converts this bit stream into 64-bit slices, which implies that a new slice arrives every 6.4ns at 10Gb/s, and every 650ps at 100Gb/s.

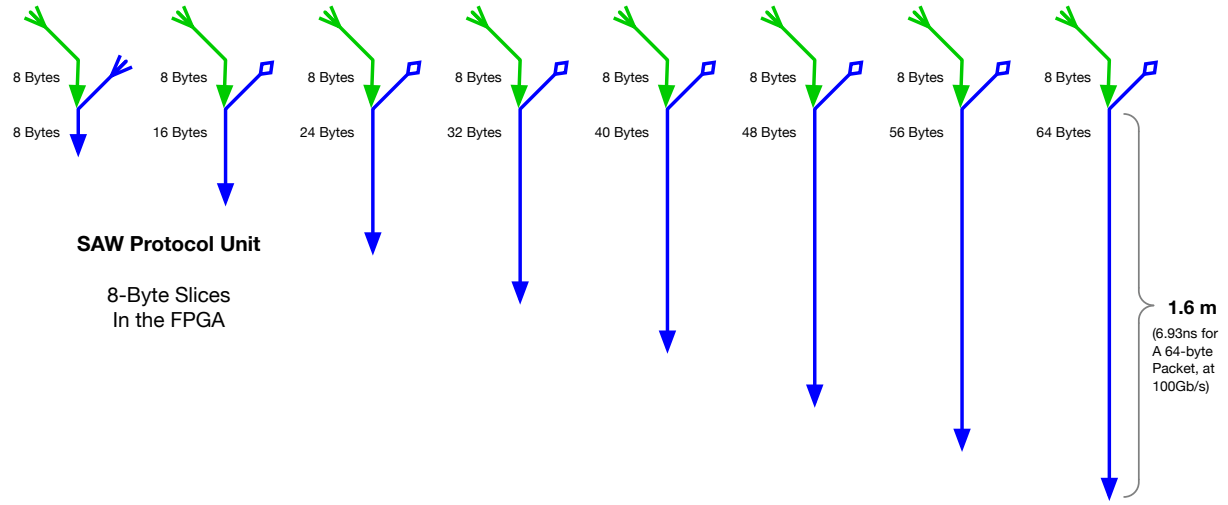
We could choose Flow-control units (flits) of 1 slice, or 1 frame. We choose 1 frame for the Stop and Wait (SAW) flow control unit, because (a) we know the receiver can receive up to 8 slices[1] without having to insert another inter-record-gap (IRG) or preamble, and (b) we can pre-empt the sending of routine status/maintenance information (such as liveness tensor), by using the equivalent of an Ethernet Jamming signal (part of the SAW protocol (in Slice-Engine.pdf).

The speed of light in copper is 2.3e+08 ms^{-1}. These numbers translate to *time of* bits on the wire, as shown in the figure below, which translates to *distance occupied* by those bits on the wire, as shown in the figures below for the slices within an Ethernet Frame from 1 Gigbit to 1 Terabit.

At 100Gbit a 64B frame occupies 1.6m on the wire, and two 64B frames will occupy 3.2m. Unlike with long cables, there is no point in stuffing more than one frame down the wire at a time. Flow control units (Flits) can use the Stop And Wait (SAW) protocol without any loss of bandwidth efficiency on the wire. Links manage flow control, there is no 'delayed' PFC type (PAUSE FRAME) mechanism. Stop and Wait (SAW) does it for you (See FAQ at end).

[1] A fixed packet size 1500 bytes would not optimize for ultra-low latency, and would not match the cache line size

Stop and Wait Wire Occupation Time, and Length



At 100Gbit with 20 bytes of frame overhead an 8 slice (64B frame) occupies 1.6m on the wire. The Stop and Wait (SAW) protocol allows two frames to be in flight *simultaneously*: one transmitted and one received (full-duplex) on the same cable. Even with one slice only, when you include the Ethernet overhead, that's over half a metre at 100Gbits. From an engineering perspective, there is an opportunity to optimize a next generation of Ethernet for short cables.

Slice Transmission Times From 1 Gigabit to 1 Terabit Ethernet (in ns)

Over-head	Slices	Slice Size (B)	Total (Bytes)	Total (Bits)	Bits after encoding	Total Bit Times (ns) (Assumes 1Gbit is 1ns per bit)									
						Ethernet Bit Transmission (from 1Gbit to 1 TBit									
20		8			1.03125	1	10	25	40	50	100	200	400	800	1000
One Bit Time					1.00000	1.0000	0.1000	0.0400	0.0250	0.0200	0.0100	0.0050	0.0025	0.0013	0.0010
20	1	8	28	224	231.0	231.0	23.100	9.240	5.775	4.620	2.310	1.155	0.578	0.289	0.231
20	2	16	36	288	297.0	297.0	29.70	11.88	7.43	5.94	2.97	1.485	0.743	0.371	0.297
20	3	24	44	352	363.0	363.0	36.30	14.52	9.08	7.26	3.63	1.815	0.908	0.454	0.363
20	4	32	52	416	429.0	429.0	42.90	17.16	10.73	8.58	4.29	2.145	1.073	0.536	0.429
20	5	40	60	480	495.0	495.0	49.50	19.80	12.38	9.90	4.95	2.475	1.238	0.619	0.495
20	6	48	68	544	561.0	561.0	56.10	22.44	14.03	11.22	5.61	2.805	1.403	0.701	0.561
20	7	56	76	608	627.0	627.0	62.70	25.08	15.68	12.54	6.27	3.135	1.568	0.784	0.627
20	8	64	84	672	693.0	693.0	69.30	27.72	17.33	13.86	6.93	3.465	1.733	0.866	0.693
20	16	128	148	1184	1221.0	1,221.0	122.10	48.84	30.53	24.42	12.21	6.105	3.053	1.526	1.221

Slice Transmission Distances (in m)

						SOL	ns Conversion	m conversion							
	Slices	Slice Size (B)	Total (Bytes)	Total (Bits)	Bits after encoding	2.3E+08	1E-09	1		Distance travelled for these bit times (in m)					
		8			1.03125	1	10	25	40	50	100	200	400	800	1000
One Bit distance					1.00000	0.23000	0.02300	0.00920	0.00575	0.00460	0.00230	0.00115	0.00058	0.00029	0.00023
	1	8	8	64	66	53.130	5.313	2.125	1.328	1.063	0.531	0.266	0.133	0.066	0.053
	2	16	16	128	132	68.310	6.831	2.732	1.708	1.366	0.683	0.342	0.171	0.085	0.068
	3	24	24	192	198	83.490	8.349	3.340	2.087	1.670	0.835	0.417	0.209	0.104	0.083
	4	32	32	256	264	98.670	9.867	3.947	2.467	1.973	0.987	0.493	0.247	0.123	0.099
	5	40	40	320	330	113.850	11.385	4.554	2.846	2.277	1.139	0.569	0.285	0.142	0.114
	6	48	48	384	396	129.030	12.903	5.161	3.226	2.581	1.290	0.645	0.323	0.161	0.129
	7	56	56	448	462	144.210	14.421	5.768	3.605	2.884	1.442	0.721	0.361	0.180	0.144
	8	64	64	512	528	159.390	15.939	6.376	3.985	3.188	1.594	0.797	0.398	0.199	0.159
	16	128	128	1024	1056	280.830	28.083	11.233	7.021	5.617	2.808	1.404	0.702	0.351	0.281

Optimize Latency, not Bandwidth. No Queues or Buffering Needed

For low-latency applications, queues, or any kind of buffering, is detrimental. The Stop and Wait (SAW) protocol must be one unit. Ethernet compatibility would suggest 64 bytes. Modern FPGA implementations, which process one 8-byte slice at a time, could also have a SAW unit of one slice[2].

Optimizing for bandwidth is what the industry has done for decades. But it's no longer bandwidth that is the scarce resource anymore, it's low-latency. For an ultimate low-latency transaction protocol, the engineering tradeoffs are very different. Instead of thinking in terms of arbitrary length buffers, we think instead in terms of registers (8 byte – 64bits), which match the slices processed by the FPGAs. Each 'register set' is matched to some number of round-trips (1, 2, 4, 8) to complete the protocol. See State-Machine-Specification.pdf.

It is difficult to over-emphasize this point. Ultra-Low latency requires SAW units that are smaller than the cable length. When cable lengths are short, there is no point stuffing more than one unit down the wire at once, all you will do is over-run the SERDES and it will drop packets. At 100Gbits/s (100bits/ns), cable lengths are on the order of 1.6m. Even at 1Tbit (1000 bits/ns) cable occupation times for 64byte frames are still 16 cm ($\sim$ 6 inches), which is easily long enough to connect adjacent cells in a rack.

Even if the SERDES had enough capacity and buffering, there is a limit of around 40 Gb/s because the limitations of the host processor memory hierarchy on the other side of the PCIe bus.

The illusion of Limited Bandwidth on Short Cables

The idea that cables must be short or be bandwidth limited is a misconception. Copper cables are not bandwidth limited because of length, they 'appear' that way because they are (a) not correctly terminated, and (b) the path variations in cross-sectional dimensions, dielectric permittivity, etc., cause Zo to be non-uniform.

Each variation in Z_0 creates reflections, which add up in smaller and smaller 'steps' to 'look like' capacitive rise times growing exponentially to an asymptote of voltage. You can see this on a high bandwidth HP or Tektronix oscilloscope. In a 'good' (uniform, properly terminated) transmission line, there is no imaginary component to the impedance; it should really be called R_0 (characteristic resistance), not Z_0.

You can see this misconception being played out in the History of Ethernet Cables. It's clear that this stubbornly persistent illusion keeps gets getting discredited each time the industry improves the quality (i.e. transmission line characteristics) of their cables.

SPICE looks at the problem under the wrong (non-relativistic) light. Oliver Heaviside showed us that electromagnetic energy travels in the medium outside the conductors, not via the electrons in the conductors. This is what made Nikola Tesla's AC so incredibly important. The 'illusion' of ringing of a rising signal, or the inverse exponential growth of a signal, are symptoms of the transmission line being over- or under-terminated. When the transmission line is correctly terminated, with a resistance, you will get an optimal signal.

We do not want a DC signals, even through copper cables. Transformers are used in Ethernet to isolate the Ground reference. This also makes them less susceptible to common mode interference. The lack of DC signal for detection of link status is overcome by the natural high frequency of the alternating causality tokens. See the Dual SAW Spekkens document for details.

What this means for our product strategy is:

- We don't need to worry about there being some fundamental limit to bandwidth. We blew past 10 Mbit, 100Mbit, 1Gbit, and 10Gbit, and now 100Gbit, and this will continue in the future.
- Cable length effects are a function of attenuation due to resistance (partly skin effect), which means that shorter cables will (in principle) require less energy to drive them to a threshold a receiver can reliably detect. Reducing energy consumption is an additional cost/ manufacturing/ OpEx advantage.
- The rest of the industry is only just coming to terms with the reality that optical cables may be unnecessary (certainly within racks). Even the 'potential' to use optical by requiring SFP or QSFP physical adapters unnecessarily increases costs.

[2]There are some constraints in current implementations of the SERDES, which we may have to live with. From a business perspective it would be much better to use what is available off the shelf right now in terms of available technology, than to try and change things. Having said that, because we need only be compatible with other SmartNICs like ourselves, and not switches or routers are usually optimized for bandwidth, we have fewer constraints to deal with than what might otherwise be imagined.

Ethernet Jam Signal

A jam signal was part of the original Ethernet CSMA/CD standard. In a collision domain (Bus, Hub) stations that saw collisions continued to transmit a jam signal for a period of time to reset timers and wait/listen for quiet before sending again. The durations were set based on the length of the bus, and the number of bits on the wire (which also determined the inter-record gap).

This is no longer needed in principle in point to point links, or is it? In principle, in a full-duplex pair of transmit and receive wires, we could imagine that there is no need for a Jam signal.

Half duplex is when a transmitter and receiver take turns to occupy the transmission medium (collision domain) to communicate.

Full duplex is when there are two separate mediums, and both can transmit and receive simultaneously. This is what allows human beings to talk on top of one another when they are in the same room or on a Zoom call.

The Stop and Wait (SAW) protocol is *alternating duplex*. By tuning the amount of data on the wire to be the same as the flow control unit (which is typically 128 bytes in conventional networks) we can enhance the reliability of the network links by preventing the SERDES from ever being overridden.

Alternative Axioms (Optimizing for Latency, not Bandwidth)

1. The SERDES can handle one Stop and Wait (SAW) Unit of protocol.
    a) The SAW unit is *at most* one 64-byte frame.
    b) The SAW unit is *at least* one 8-byte slice.
2. It is more economical to accommodate bit rates in the SERDES if you take the available chip bandwidth and separate it out into 8 *independent* channels.
    a) 8 independent channels of 100Gbits each instead of one channel of 800Gbits.
    b) 8 independent channels of 50 Gbits instead of one 400 Gbits.
    c) 8 independent channels of 25Gbits instead of one channel of 200 Gbits.
3. The SERDES never drops packets. If it can't take any more, it doesn't send it's SAW token. The sender promises to not send a packet if it doesn't have the token.
4. Failures (loss of events) are not handled by timeouts. They are managed by a higher level (event harvesting protocol) in a multiport SmartNIC, which has access to liveness events on its other links. See Liveness-Tensor.pdf for details.

Optimizing for Ultra-Low Latency and Resilience

> Take care of the nanoseconds and the gigabits will take care of themselves.

Unlike the situation with long cables, there is no point in stuffing more than one (or two) frames down the wire at a time. When the cables between cells in a rack are a fraction of a metre, or between chips in a chip interconnect (on the order of cm or mm), flow control can occur using the Stop And Wait (SAW) protocol without any loss of bandwidth efficiency on the wire.

It is far more important to *not* discard packets (deliberately or otherwise), than it is to optimize the bitrate available in the latest technology.

The Ethernet industry has so far been unable to solve two important problems. (a) Congestion. Lossless (IEEE 802 Nendica Report: The Lossless Network for Data Centers). (b) the insecure nature of Any to Any (A2A) protocols.

Strategies to Overcome SERDES limitations with increasing bit-rates

We identified several layers of strategy to overcome the SERDES limitations compatibly with our 'ultra-low latency' use case.

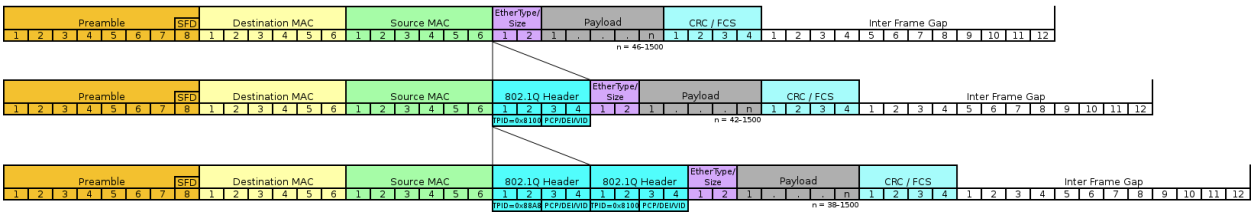
Pause Frames (hanging on to the Colored Petri Token)

- For accessing SRAM
- For updating the routing table (or healing the tree around failures)
- For accessing large DRAMs
- Anything else to pause frames (by hanging on to the token)?

Ethernet Frame Compatibility

The Phenomenal success of Ethernet is partially an Axe evolution (swap out the head for a better one, but keep the handle. Swap out the handle when it wears out, but keep the head). Pretty much the only thing that has been maintained as a constant is the interface – the Ethernet Frame.

We would like to continue to do that within an axiomatic framework that enables Ethernet to evolve, in metamorphosis, as it did over the years, from a bus, to a hub (3-Com) to a router (Cisco). Speeds increased from 10Mbit to 100Mbit, and now 20 years later, from 10Gbit to 100 Gbit with 1Tbit on the horizon. The only thing limiting the potential future 'serial bandwidth' of Ethernet is the imagination of engineers and product designers. We know from Oliver Heaviside that there are no fundamental limits in the physics of transmission lines. The flow rate is limited by the SERDES, not by the number of packets you can stuff on the wire (with long cables).



Indistinguishability of Events

Manchester code

There is a rich history of highly creative engineering beginning with Bob Metcalf's original Ethernet.

Encoding each bit as low voltage or high voltage (corresponding to whether the driver is inverted or not) on the wire for 'equal periods'. Is a 'self-clocking' scheme invented at Manchester University. The exclusive-OR boolean function $\oplus$ of a clock and a data signal has no DC component, and is 'galvanically isolated'. This means that binary information (0 or 1) is represented as *transitions* rather than voltate *levels*[3]. There is a pairing of one transition for each binary digit transmitted, with an intrinsic clock recovery per bit period.

What's missing from a protocol perspective, is a robust notion of reversibility[4]. An important innovation in the next evolution of protocols, is being able to exploit reversibility to engineer recoverable systems. A simple perspective is to 'just use' Legacy Protocols such as TCP/IP and build reversible state machines at each end. To understand why this will always be insufficient, we need to explore what Shannon taught us about Information.

[3] See Paul Borrill's, PhD Thesis, 1986 University College London.
[4] Quantum and Classical Physics are Reversible.

Information is a Surprise

Information is a highly perishable commodity. What we call information in memory and on disks might be more appropriately called knowledge. Bits arriving off the wire resolve the uncertainty of *when* a question is answered.

1. Once we have received (*and stored*) a bit in memory, it is no longer a surprise. This is reflected in Manchester encoding – with information conveyed by the 'transitions', not the 'levels'.

2. Information is always paired (question, binary answer). If you send information out without having a relationship with a receiver, you are sending noise, which often gets confused with entropy (the loss of information) from the transmitters' perspective. If you receive information from an unknown source that you don't have a relationship with, you are receiving noise.

3. Some notion of 'common knowledge' is required so that both end's of a channel agree on what the question/answer pairings mean *apriori*. We can think of information as the derivative of knowledge (or knowledge as the integral of information).

4. Receiving an information bit 'resolves the uncertainty' of the yes/no question. But if we remember this binary answer (by, for example, storing it in memory), then can we call it knowledge? Perhaps we can, but we need to then acknowledge Robert Spekkens' knowledge balance principle.

5. We cannot simulate quantum entanglement with a state machine[5]. You can simulate quantum entanglement, and many of it's previously thought to be quantum-only properties (such as no-cloning and teleportation) with a Petri net and two conserved tokens. See Dual-SAW-Spekkens.pdf.

Maintaining liveness and synchronizing processes in network channels is a perennial challenge when packets can be dropped, reordered, duplicated or delayed. Conventional switched networks require protocol stacks and applications to use timeouts and retries to maintain liveness. This makes exactly-once semantics impossible and precipitates retry storms which lead to unbounded tail latency and transaction failure.

A superior solution is to employ individual direct channels with the Stop and Wait (SAW) protocol to maintain flow control, Colored Petri Nets1 (CPTs) to process different actions (for different orientations), and the Spekkens' Toy Model (STM) to manage epistricted consistency for distributed metadata.

Information Must Be Paired

One clue that information must be paired comes from the need for *disparity codes* used in Ethernet. Just as we will find 'drift' in the DC level and leakage of the gavanic isolation of networks, we will find that 'information' must also be paired when going through DC isolation elements (capacitors or transformers). Balancing risetimes and falltimes is insufficient, timeconstants may be different and more sophisticated techniques may be needed, such as in Interlaken, where the entire stream of bits are inverted. Transition Balancing is tricky enough, but relatively blunt instruments such as disparity code tricks described above are still not accurate enough to deal with precise quantities of information, such as those required in financial systems.

But balancing ones and Zero's are also insufficient from the 'Knowledge Balance Principle' Because links break, NIC's fail, Chips fail, firmware crashes and has to be reset, and device drivers (and operating systems) fail and need to be restarted. If that was all that we had to deal with, then it would be more difficult to overcome the 'good enough' battle cry of business folk who would rather expend resources in selling what they have rather than solving a fundamental problem. This is not an unreasonable business perspective when the problem looks small (on the surface) appears to be hopeless (no solution in sight).

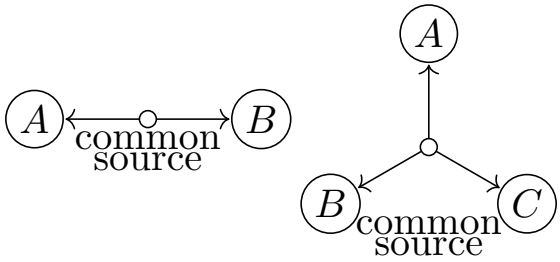
But it's much worse than this: It doesn't take much thinking to recognize that the established paradigm of the networking industry, to Drop, Reorder, Duplicate and Delay packets will amplify this inability to accurately account for event pairings, such that these failures, instead of happening some 10^{-15}, instead are happening constantly, with some large fraction of the traffic being subject to this 'information loss'.

[5] A state machine is a Petri Net with one conserved token

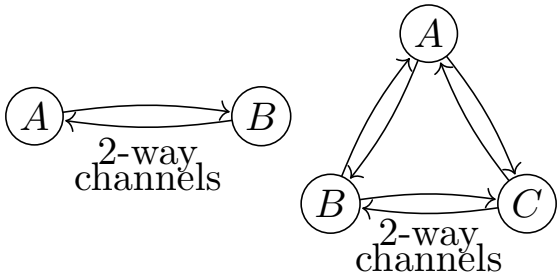
Pairing Information, Not Transitions

Unlike disparity codes and long term DC balance, information can be accounted for in pairs by setting up the system of question/answer to grow and shrink as the protocol proceeds through a transaction. We claim that information can be balanced if and only if:

1. The system has Local Operations and Shared Randomness (LOSR) as a resource[6]. A tripartite (Alice, Bob, Charlie). See Dual-SAW-Spekkens.pdf
2. Hidden Nonlocality. The system starts from a quiescent state of 'alternating causality'. The Stop and Wait (SAW) protocol provides
3. Network structures are critically relevant. See Figures below (Taken from Schmidt et al).

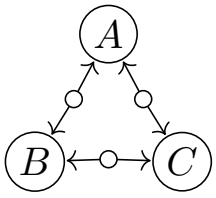


(a) A common source shared by all parties.

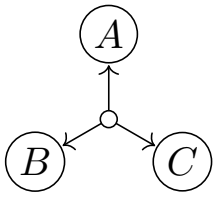


(b) Two-way channels between all pairs of parties.

Figure 1: Common source and two-way channel networks involving two and three parties.



(a) The triangle network.



(b) The trine network.

Figure 2: The triangle network and the trine network.

Nature has a built-in Handshake

We know from Bell state measurements that correlations can appear faster than the speed of light (one way non-local effects). But signaling of information requires a prepare-commit handshake. We take advantage of this insight in the design of an 'Information balancing' protocol for Ethernet. See: Slice-Engine.pdf and State-Machine-Specification.pdf

[6] Developing a new branch of entanglement theory.

Bandwidth

The industry continues to march down the path of increasing bandwidth. We've gone from 10 Mbit/s to 100 Mbit/s in Bob Metcalf's world, to 10 Gbits a decade ago, 100 Gbit/s today, and 1 Terabit/s on the horizon[7]. But these bandwidth increments are conflated with the architecture of networks

Latency

Reliability

Confinement

Any to Any (A2A) networks, are fundamentally vulnerable to attack. This is reflected in the never-ending cost in personnel to administer IT systems today. It doesn't matter if these systems are on-prem, or in the cloud. There is no limit to the amount of time and effort human beings need to pour into managing these systems.

Cost

Unique Identifiers

The IEEE has made money selling 48-bit numbers. This has worked well, but requires the intervention (or checking) by human beings to guarantee that the 48-bit identifier is unique across all of space and time. Two machines having the same identifier

The alternative, to have locally administered numbers creates a central repository, with its associated failure mode and bottlenecks which inhibit scalability.

While there is always at least one transition per bit period to enable clock recovery at the receiver, this is a 'forward-in-time-only' concept

This is where the physics perspective comes in. Trying to do a reversible state machine at two ends of a TCP/IP connection requires

FAQ

[Q1] How is Stop and Wait (SAW) different to pause frames in PFC?

[A1] We follow the principle of Promise Theory. PFC assumes an imposition.

[A1] SAW is 'permission' – the other side will send a packet when they ARE ready. PFC is 'forgiveness': other side will send you a Pause Frames when they are NOT ready (similar to the XON-XOFF protocol).

References

1. From: DCFIT: Initial Trigger-Based PFC Deadlock Detection in the Data Plane

   "Causality-loop primitive: The causality-loop can be determined when the causality chain of pause frames has visited the same port of the same switch twice."

2. Improving performance of backpressured packet networks by integrating with an end-to-end congestion control algorithm.

3. Selective backpressure control for multistage switches

[7] Synopsys: An Insight into the World of 224Gbps Electrical Interface..